# *Singly Linked List – Creation of Empty List –Space Complexity and Algorithm.*

## *Program* :

```
#include <iostream>
using namespace std;
typedef struct ListNode
{
    int data;
    struct ListNode *next;
} Node;

Node *head;

void createEmptyList(Node **head)
{
    *head = nullptr;
}

int main()
{
    createEmptyList(&head);
}
```

## ***Space Complexity***

*The space complexity of a program refers to the amount of memory space required by the program as a function of the input size. In this particular program, the space complexity is determined by the following factors:*

1. *Node Structure: The ListNode structure is defined to hold an integer (int data) and a pointer to the next node (struct ListNode * next). The space required for each node is constant, i.e., $O(1)$.*

2. $Head\ Pointer$: $The\ head\ pointer\ is\ a\ global\ variable\ that\ points\ to\ the\ start\ of\ the\ linked\ list. It\ requires\ a\ constant\ amount\ of\ space\ , i.e., O(1)$.

3. $Function\ Call$: $The\ createEmptyList\ function\ takes\ a\ pointer\ to\ a\ pointer\ as\ an\ argument. The\ space\ required\ for\ this\ function\ call\ is\ also\ constant,\ i.e., O(1)$.

$Since\ the\ program\ only\ initializes\ an\ empty\ list\ and\ does\ not\ create\ any\ nodes\ dynamically, the\ overall\ space\ complexity\ of\ this\ program\ is\ O(1)$.

---

$1. Input\ Space\ Complexity$: $O(1)$

$The\ input\ space\ is\ the\ memory\ required\ to\ store\ the\ parameters\ passed\ to\ the\ function.$

$In\ this\ case, the\ createEmptyList\ function\ takes\ a\ single\ argument$:

$Node\ **\ head$: $This\ is\ a\ pointer\ to\ a\ pointer. The\ memory\ required\ to\ store\ a\ pointer\ (or\ a\ pointer\ to\ a\ pointer) is\ a\ fixed, constant\ size\ (e.g., 8\ bytes\ on\ a\ 64-bit\ system)$.

$Since\ the\ function's\ input\ is\ always\ a\ single\ address\ of\ a\ constant\ size,\ the\ input\ space\ complexity\ is\ O(1). The\ function\ isn't\ being\ given\ a\ list\ of\ n\ elements\ to\ process;\ it's\ just\ being\ given\ the\ address\ of\ the\ head\ pointer\ itself.$

$2. Auxiliary\ Space\ Complexity$: $O(1)$

$The\ auxiliary\ space\ is\ the\ extra\ memory\ the\ function\ uses\ during\ its\ execution\ , excluding\ the\ input\ space.$

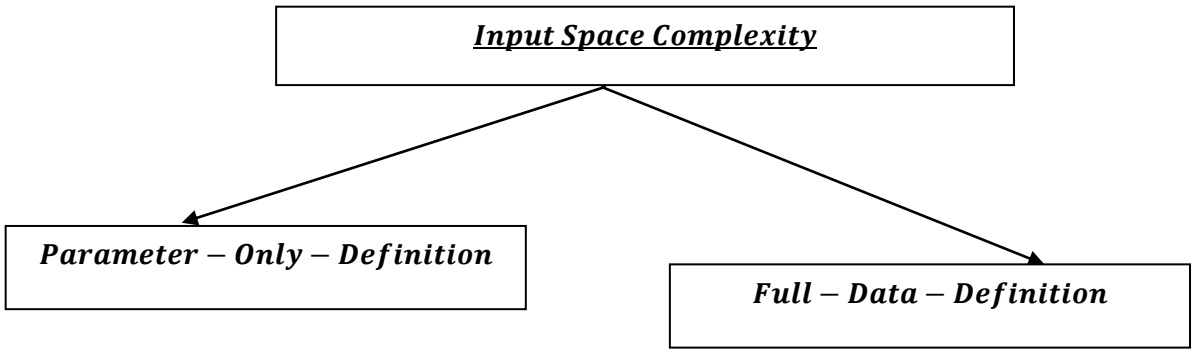
$The\ createEmptyList\ function\ is\ very\ simple$:

$It\ performs\ one\ operation$: $*\ head\ =\ nullptr;$.

$It\ does\ not\ declare\ any\ new\ variables, create\ any\ new\ data\ structures, or\ use\ recursion.$

$Because\ the\ function\ uses\ no\ additional\ memory\ to\ perform\ its\ task,\ its\ auxiliary\ space\ complexity\ is\ constant, or\ O(1)$.

---




$Parameter - Only\ Definition\ (O(1))$: $This\ view\ considers\ only\ the\ memory\ used\ by\ the\ parameters\ themselves. Since\ Node\ **\ head\ is\ a\ single\ pointer - to - a - pointer, its\ size\ is\ fixed, making\ the\ complexity\ O(1)$.
$This\ is\ the\ most\ literal\ interpretation$.

$Full - Data\ Definition\ (\Theta(n))$: $This\ is\ the\ more\ common\ and\ practical\ convention\ in\ algorithm\ analysis. It\ includes\ the\ entire\ data\ structure\ accessible\ from\ the\ input\ parameters. If\ head\ pointed\ to\ a\ list\ with\ n\ nodes, the\ input\ space\ would\ be\ \Theta(n)$.

---

# ***Algorithm of Creation of Empty List***

***Step* 1**: ***DEFINE STRUCTURE***

- ***Define a structure ListNode with two members***:

  - ***An integer data to store the value of the node.***
  - ***A pointer next of type ListNode $*$ to point to the next node in the list.***

***Step* 2**: ***Type Alias***

- ***Use typedef to create an alias Node for struct ListNode to simplify the code.***

***Step* 3**: ***DECLARE GLOBAL POINTER***

- ***Declare a global pointer head of type Node $*$ to keep track of the head of the list.***

***Step* 4**: ***CREATE FUNCTION***

- ***Define a function ``createEmptyList`` that takes a double pointer Node $**$ head as an argument.***

- ***SET HEAD $:=$ NULLPTR***

***Step* 5**: ***Call Function $\rightarrow$ createEmptyList*** .

---